

Practical-6

Aim:

To perform E-Commerce Sales Analysis using Apache Spark DataFrames by loading sales data, calculating revenue, and generating category-wise, product-wise, and city-wise analytical insights.

Problem Definition:

You are a Data Analyst at ShopSmart Analytics Pvt. Ltd. The company operates a large-scale e-commerce platform that processes thousands of transactions every day. Management wants to identify high-performing product categories, top-selling products, and revenue contribution by city. Traditional spreadsheet tools are unable to efficiently process the growing volume of transaction data. Therefore, the company has adopted Apache Spark for scalable analytics. Your task is to design a Spark-based analytical solution to process sales data and generate business insights.

Objectives:

1. To load e-commerce sales data into a Spark DataFrame.
2. To display and validate the schema and data types of the dataset.
3. To create a Revenue column using $\text{Quantity} \times \text{Price}$.
4. To calculate category-wise revenue.
5. To identify top-selling products based on quantity sold.
6. To calculate and sort city-wise revenue.
7. To generate analytical summaries from the sales data.
8. To interpret the results and provide suitable business recommendations.

Objective/Output:

Tasks:1 Load sales data into a Spark DataFrame.

```

gitanshu@Parth:~$ py --version
Command 'py' not found, but can be installed with:
sudo apt install pythonpy
gitanshu@Parth:~$ python3 --version
Python 3.12.3
gitanshu@Parth:~$ python3 -c "import pyspark; print(pyspark.__version__)"
Traceback (most recent call last):
  File "<string>", line 1, in <module>
ModuleNotFoundError: No module named 'pyspark'
gitanshu@Parth:~$ mkdir -p ~/spark_practical6
gitanshu@Parth:~$ cd ~/spark_practical6
gitanshu@Parth:~/spark_practical6$ pwd
/home/gitanshu/spark_practical6
gitanshu@Parth:~/spark_practical6$ ls
gitanshu@Parth:~/spark_practical6$ nano ecommerce_sales.csv
gitanshu@Parth:~/spark_practical6$ ls
ecommerce_sales.csv
gitanshu@Parth:~/spark_practical6$ cat ecommerce_sales.csv
OrderID,Date,Product,Category,Quantity,Price,City
1001,2026-01-05,Laptop,Electronics,2,55000,Ahmedabad
1002,2026-01-07,Mouse,Electronics,5,800,Vadodara
1003,2026-01-10,Chair,Furniture,3,4500,Surat
1004,2026-01-15,Keyboard,Electronics,4,1500,Ahmedabad
1005,2026-01-20,Desk,Furniture,2,7000,Vadodara
1006,2026-02-02,Mobile,Electronics,3,25000,Surat
1007,2026-02-08,Sofa,Furniture,1,30000,Ahmedabad
1008,2026-02-15,Headphones,Electronics,6,2000,Vadodara
1009,2026-02-20,Table,Furniture,4,6000,Surat
1010,2026-03-01,Laptop,Electronics,3,55000,Ahmedabad
1011,2026-03-05,Mouse,Electronics,8,800,Surat
1012,2026-03-10,Chair,Furniture,5,4500,Vadodara

```

2.Display the schema of the dataset.

OrderID	Date	Product	Category	Quantity	Price	City
1001	2026-01-05	Laptop	Electronics	2	55000	Ahmedabad
1002	2026-01-07	Mouse	Electronics	5	800	Vadodara
1003	2026-01-10	Chair	Furniture	3	4500	Surat
1004	2026-01-15	Keyboard	Electronics	4	1500	Ahmedabad
1005	2026-01-20	Desk	Furniture	2	7000	Vadodara
1006	2026-02-02	Mobile	Electronics	3	25000	Surat
1007	2026-02-08	Sofa	Furniture	1	30000	Ahmedabad
1008	2026-02-15	Headphones	Electronics	6	2000	Vadodara
1009	2026-02-20	Table	Furniture	4	6000	Surat
1010	2026-03-01	Laptop	Electronics	3	55000	Ahmedabad
1011	2026-03-05	Mouse	Electronics	8	800	Surat
1012	2026-03-10	Chair	Furniture	5	4500	Vadodara

3.Validate data types of all columns.

```

root
|-- OrderID: integer (nullable = true)
|-- Date: date (nullable = true)
|-- Product: string (nullable = true)
|-- Category: string (nullable = true)
|-- Quantity: integer (nullable = true)
|-- Price: integer (nullable = true)
|-- City: string (nullable = true)

```

4. Create a new Revenue column using Quantity x Price.

```

OrderID -> int
Date -> date
Product -> string
Category -> string
Quantity -> int
Price -> int
City -> string

```

TASK 4: REVENUE COLUMN

OrderID	Date	Product	Category	Quantity	Price	City	Revenue
1001	2026-01-05	Laptop	Electronics	2	55000	Ahmedabad	110000
1002	2026-01-07	Mouse	Electronics	5	800	Vadodara	4000
1003	2026-01-10	Chair	Furniture	3	4500	Surat	13500
1004	2026-01-15	Keyboard	Electronics	4	1500	Ahmedabad	6000
1005	2026-01-20	Desk	Furniture	2	7000	Vadodara	14000
1006	2026-02-02	Mobile	Electronics	3	25000	Surat	75000
1007	2026-02-08	Sofa	Furniture	1	30000	Ahmedabad	30000
1008	2026-02-15	Headphones	Electronics	6	2000	Vadodara	12000
1009	2026-02-20	Table	Furniture	4	6000	Surat	24000
1010	2026-03-01	Laptop	Electronics	3	55000	Ahmedabad	165000
1011	2026-03-05	Mouse	Electronics	8	800	Surat	6400
1012	2026-03-10	Chair	Furniture	5	4500	Vadodara	22500

5. Calculate category-wise revenue.

Category	Total_Revenue
Electronics	378400
Furniture	104000

6. Identify top-selling products based on quantity sold.

Product	Total_Quantity_Sold
Mouse	13
Chair	8
Headphones	6
Laptop	5
Table	4
Keyboard	4
Mobile	3
Desk	2
Sofa	1

7. Calculate city-wise revenue.

City	Total_Revenue
Ahmedabad	311000
Surat	118900
Vadodara	52500

8. Sort categories based on revenue generated.

Category	Total_Revenue
Electronics	378400
Furniture	104000

9. Display analytical summaries.

```
Total Orders: 12
Total Revenue: 482400
Total Quantity Sold: 46
```

10. Interpret business insights and recommendations.

```
Highest Revenue Category: Electronics
Top-Selling Product: Mouse
Highest Revenue City: Ahmedabad
```

Key Questions /Analysis

Q1. Why are Spark DataFrames preferred over traditional spreadsheets for large-scale analytics?

Spark DataFrames are preferred because they can process **large volumes of data efficiently** using distributed computing. They can divide data and processing tasks across multiple CPU cores or machines. Spreadsheets are generally more suitable for smaller datasets and can become slow or difficult to manage as data size increases.

Main advantages of Spark DataFrames:

- Handle large datasets.
- Faster processing through parallelism.
- Distributed processing across multiple machines.
- Support SQL-like operations such as `groupBy()`, `filter()`, and `orderBy()`.
- Can process data from CSV, JSON, databases, and other sources.

Q2. What is the role of the `groupBy()` operation in data aggregation?

`groupBy()` is used to **group records having the same value in a particular column** so that aggregate operations can be performed on each group.

For example:

```
df.groupBy("Category").agg(  
    sum("Revenue").alias("Total_Revenue")  
)
```

This groups all records according to Category and calculates the total revenue for each category.

Example:

Electronics → Total Revenue

Furniture → Total Revenue

Thus, `groupBy()` is useful for **category-wise, city-wise, product-wise, and other aggregated analysis**.

Q3. Explain the difference between transformations and actions in Spark.

Transformations	Actions
Create a new DataFrame/RDD from existing data	Produce a result or output
Executed lazily	Trigger actual execution
Do not immediately execute the computation	Start Spark job execution
Examples: <code>filter()</code> , <code>select()</code> , <code>withColumn()</code> , <code>groupBy()</code>	Examples: <code>show()</code> , <code>count()</code> , <code>collect()</code> , <code>first()</code>

Q4. How does Spark improve scalability for e-commerce analytics?

Spark improves scalability by using **distributed and parallel processing**.

Large e-commerce datasets can be divided into partitions and processed simultaneously.

For example:

Large E-Commerce Dataset

↓

Spark Cluster

/ | \

Partition Partition Partition

↓ ↓ ↓

Process Process Process

\ | /

Combined

Result

Therefore, Spark can handle increasing numbers of transactions, customers, products, and orders more effectively than traditional spreadsheet-based analysis.

Q5. Which category contributes the highest revenue and what business action can be taken?

For the **given dataset**, the category with the highest revenue is **Electronics**.

The company can:

- Maintain sufficient inventory of high-revenue electronic products.
- Analyze which electronic products generate the most revenue.
- Use targeted promotions for electronic products.
- Monitor customer demand and stock levels.
- Continue analyzing Furniture separately to identify opportunities for improvement.

Supplementary Problems:

Find the top 3 revenue-generating cities.

1. TOP 3 REVENUE-GENERATING CITIES	
City	Total_Revenue
Ahmedabad	311000
Surat	118900
Vadodara	52500

Calculate average revenue per order.

2. AVERAGE REVENUE PER ORDER	
Average_Revenue_Per_Order	
	40200.0

Add a discount column and recompute revenue.

3. DISCOUNT AND FINAL REVENUE

OrderID	Product	Quantity	Price	Revenue	Discount	Final_Revenue
1001	Laptop	2	55000	110000	11000.0	99000.0
1002	Mouse	5	800	4000	400.0	3600.0
1003	Chair	3	4500	13500	1350.0	12150.0
1004	Keyboard	4	1500	6000	600.0	5400.0
1005	Desk	2	7000	14000	1400.0	12600.0
1006	Mobile	3	25000	75000	7500.0	67500.0
1007	Sofa	1	30000	30000	3000.0	27000.0
1008	Headphones	6	2000	12000	1200.0	10800.0
1009	Table	4	6000	24000	2400.0	21600.0
1010	Laptop	3	55000	165000	16500.0	148500.0
1011	Mouse	8	800	6400	640.0	5760.0
1012	Chair	5	4500	22500	2250.0	20250.0

Find the least-performing product category.

4. LEAST-PERFORMING PRODUCT CATEGORY

Category	Total_Revenue
Furniture	104000

only showing top 1 row

Generate a monthly revenue summary.

5. MONTHLY REVENUE SUMMARY

Month	Total_Revenue
1	147500
2	141000
3	193900

Tools and Technologies:

- **Programming Language:** Python 3
- **Processing Framework:** Apache Spark
- **Spark API:** PySpark
- **Data Processing:** Spark DataFrames

- **Input Data Format:** CSV
- **Data Source:** E-Commerce Sales Dataset
- **Data Analysis:** Spark SQL / DataFrame Operations
- **Aggregation:** groupBy(), agg(), sum(), avg()
- **Data Transformation:** withColumn(), to_date(), month()
- **Sorting:** orderBy()
- **Operating System:** Ubuntu Linux / WSL2
- **Execution Environment:** Local Spark (local[*])
- **Package Management:** Python pip
- **Virtual Environment:** Python venv
- **Development Environment:** Ubuntu Terminal / Nano
- **Dataset File:** ecommerce_sales.csv

Conclusion:

The e-commerce sales data was successfully processed using Apache Spark DataFrames. The dataset was loaded and its schema and data types were validated. A new revenue column was calculated using $\text{Quantity} \times \text{Price}$. Category-wise revenue, top-selling products, and city-wise revenue were analyzed and sorted to identify important sales patterns. The practical demonstrates how Spark DataFrames can efficiently perform large-scale data processing and aggregation for e-commerce analytics.

Github Repo:

https://github.com/Gitanshu1504/24DIT070_BDA_Practicals_2026_SHP